BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

According to the lecture, programming a mathematically perfect Table Driven Agent for complex environments like Chess is impossible due to the problem of Combinatorial Explosion. A table driven agent requires a lookup table to map every possible history of the world. If |P| represents the number of possible percepts and T represents the lifetime of the agent in steps, the table must contain |P|^T entries. As the agent's lifetime increases, this table grows exponentially and becomes impossibly large. For example, Chess has roughly 10^40 legal board states and an estimated 10^120 possible game sequences, known as The Shannon Number. Therefore, there is no hard drive in the universe large enough to store such a table, no human could write it, and no compiler could build it

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if wall_ahead:
    return left_turns[facing]
else:
    return facing
```

The if wall_ahead:- statement acts as the condition, checking the current percept to see if a wall is directly ahead. If this condition evaluates to true, the action triggered is return left_turns[facing] (turning left). Otherwise, the else statement triggers the default action of return facing (moving forward). This directly mirrors the lecture concept of Condition Action rules, where the agent maps current percepts straight to actions without using any internal memory or history

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The SimpleReflexAgent failed because simple reflex agents require a fully observable environment to work correctly. In Step 1.2, the environment was changed to be partially observable, where critical data is hidden and the agent only sees what is immediately ahead. Because the agent lacks a percept history (zero memory), it has no record of past locations and cannot recognize when it is looping through the same positions. Consequently, when trapped in a confined area, the constant condition (wall_ahead) continuously triggers the same reflexive action, forcing the agent into an unbreakable infinite loop

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The ModelBasedAgent handles the required models by maintaining an internal state dictionary and coordinate tracker. The Sensor Model is managed by recording how the agent's actions modify its state, such as updating coordinates (self.local_y += 1) and adding current positions to memory (self.visited_cells.add((self.local_x, self.local_y))). The Transition Model is handled by evaluating unobserved environmental aspects against memory, such as checking left_is_visited = (lx, ly) in self.visited_cells to reason about adjacent cells. By updating this internal model before applying condition action rules, the agent can navigate hidden barriers

Finalize and Lock Lab 02 Documentation

All questions answered.

FACULTY OF COMPUTING